

Hoeffding Adaptive Tree Regressor

Teoria, Implementazione in MOA/CapyMOA e Validazione Sperimentale

Stream Data Analytics 2025–2026

Andrea Zhang

Politecnico di Milano

2026

- 1 Motivazione
- 2 Background Teorico
- 3 Scelte di Design e Risultati
- 4 Dettagli Implementativi
- 5 Conclusioni

Perché portare hatr in CapyMOA?

Il gap

River offre un'implementazione di riferimento di HATR, ma MOA/CapyMOA non dispone di un learner nativo equivalente.

Il confronto richiede quindi di cambiare framework, evaluator e pipeline sperimentale.



Il contributo

Un port Java nativo con:

- wrapper Python compatibile con CapyMOA;
- validazione rispetto a River su 8 dataset;
- integrazione nel ciclo di vita MOA.

Prestazioni osservate

- **Training:** circa 3–8× più veloce nel benchmark;
- **Memoria:** modello generalmente più compatto.

Integrazione nel framework

- **Schema ARFF:** feature nominali riconosciute automaticamente;
- **Ecosistema MOA:** stessi stream, evaluator e learner di confronto.

Il vantaggio riguarda soprattutto il training; nell'inferenza per singola istanza può pesare l'overhead Python–JVM.

Hoeffding Tree: base per lo streaming

Hoeffding Tree (VFDT) [1] è un albero decisionale online che costruisce split usando la **Hoeffding bound**:

$$\varepsilon(n, \delta) = \sqrt{\frac{\ln(1/\delta)}{2n}}$$

Regola di split: dati i due migliori attributi a_1, a_2 (per variance reduction):

$$\text{split se } \frac{\Delta \bar{G}(a_2)}{\Delta \bar{G}(a_1)} < 1 - \varepsilon \quad \text{oppure} \quad \varepsilon < \tau$$

Grace period g : si tenta lo split ogni g istanze per ammortizzare il costo.

Split criterion — Variance Reduction

$$\Delta \bar{G}(a) = \text{Var}(S) - \sum_v \frac{|S_v|}{|S|} \text{Var}(S_v)$$

- S = istanze nel nodo
- S_v = sottoinsieme che va al ramo v
- Var = varianza campionaria ($ddof=1$, Welford)

Limitazione

Un Hoeffding Tree standard non rileva i **concept drift**: una volta creato, un nodo non viene revisionato.

ADWIN: finestra adattiva per il drift

ADWIN (ADaptive WINdowing) [2] mantiene una **finestra di dimensione variabile** sullo stream di errori.

Idea: si cerca il taglio $W = W_0 \cup W_1$ che massimizza la differenza di medie; se questa supera la soglia, la parte vecchia viene eliminata.

Criterio di taglio (Babcock et al., 2003 — variance-aware):

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| > \varepsilon_{cut}$$

$$\varepsilon_{cut} = \sqrt{2 m^{-1} \hat{\sigma}_W^2 \delta'} + \frac{2}{3} m^{-1} \delta'$$

$$m^{-1} = \frac{1}{n_0 - n_{\min} + 1} + \frac{1}{n_1 - n_{\min} + 1}$$

dove $n_0 = |W_0|$, $n_1 = |W_1|$,
 $n_{\min} = \text{minWindowLength}$ e $\delta' = \ln(2 \ln(|W|)/\delta)$.

Struttura a bucket



Ogni riga i contiene bucket di dimensione 2^i ; quando una riga è piena, il *compress* fonde i due bucket più vecchi nella riga successiva.

Meccanismo adattivo di hatr

Ogni nodo monitora l'errore con ADWIN. Un **nodo interno** può inoltre ospitare:

- 1 lo storico dell'errore $|y - \hat{y}|$
- 2 un **albero alternativo** opzionale, inizialmente assente

Ciclo di vita a ogni istanza $(\mathbf{x}, y)$:

- Predici dal nodo corrente; calcola errore
- Aggiorna ADWIN
- **Se drift rilevato** e l'errore *sta aumentando*: crea albero alternativo (foglia) alla stessa profondità
- **Se entrambi i path sono maturi** ($n_a, n_c > w_{\min}$): esegui **z-test**
- Se z-test significativo: *swap* o *pruning*

Z-test per lo swap

Confronta l'errore medio del path corrente (μ_c, σ_c^2) con quello dell'alternativo (μ_a, σ_a^2) :

$$z = \frac{\mu_a - \mu_c}{\sqrt{\sigma_a^2/n_a + \sigma_c^2/n_c}}$$

$$p = 2 \Phi(-|z|)$$

- $p \leq \alpha$ e $\mu_a < \mu_c$: **swap** (alternativo è migliore)
- $p \leq \alpha$ e $\mu_a \geq \mu_c$: **pruning** (corrente è migliore)

Predizione: finché esistono alternative, media le predizioni delle foglie raggiunte lungo tutti i path attivi.

Modalità di foglia

HATR supporta tre strategie di predizione alle foglie:

MEAN

Predice con la **media target** accumulata nella foglia (aggiornamento incrementale).

$$\hat{y} = \bar{y}_{\text{leaf}}$$

Semplice, robusto, deterministico.

MODEL

Predice con un **modello lineare online** (*Perceptron*, gradiente su perdita quadratica):

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

$$\nabla = 2(\hat{y} - y) [\mathbf{x}; 1]$$

I figli ereditano una **copia** del modello del padre.

ADAPTIVE (default)

Sceglie dinamicamente tra MEAN e MODEL tramite **faded MSE** (EWMA):

$$e_{\text{mean}} \leftarrow \gamma e_{\text{mean}} + (y - \bar{y})^2$$

$$e_{\text{model}} \leftarrow \gamma e_{\text{model}} + (y - \hat{y}_m)^2$$

Usa MEAN se $e_{\text{mean}} < e_{\text{model}}$.

Bootstrap sampling opzionale (Poisson($\lambda = 1$)) in tutte le modalità per aumentare la diversità delle foglie.

Scelte di design: configurazione implementata

Cosa è stato implementato

Solo la configurazione default di River:

- Splitter: HATEBSTObserver (Truncated E-BST)
split binari per attributi numerici
- Drift detector: ADWINDetector
- Leaf model: HAPerceptron (SGD lineare)
- `binary_split=False` hardcoded
attributi nominali → multiway branch

Le principali opzioni algoritmiche sono configurabili dal wrapper Python e tradotte in flag MOA.

Motivazione

Il port replica la configurazione standard di [River](#). Senza bootstrap produce le stesse predizioni; con bootstrap i diversi generatori casuali rendono confrontabili le metriche, non le singole predizioni.

Cosa non è stato implementato

- `binary_split=True`
split nominale binario forzato (non-default in River)
- QOSplitter
richiederebbe AdaNumMultiwayBranch per attributi numerici
- **Componenti pluggabili**
splitter, drift detector e leaf model sono fissi al default River

Stato	Esempio
Configurabile	<code>grace_period</code> , <code>delta</code> ✓
Hardcoded default	<code>splitter</code> , <code>ADWIN</code> ~
Non implementato	<code>binary_split=True</code> ✗

Setup sperimentale

Dataset ($n = 17\,000$, prequential evaluation):

Dataset	Feat.	Drift
Synthetic	5	abrupt
Bike Sharing	12	naturale
Fried	10	nessuno
Hyper (lento)	10	graduale
HyperFast	10	rapido
FriedDrift	10	abrupt
RBFReg	10	graduale
ElecDemand	7	naturale

Cap a 17 000 ist. Bike/ElecDemand: reali.
Hyper/HyperFast: generatore.

Configurazioni:

- *Deterministic*: mean, no bootstrap
- *Default*: adaptive, bootstrap=True

Iperparametri comuni:

- grace_period=50, $\delta = 10^{-7}$, $\tau = 0.05$
- adwin_delta=0.002, switch_significance=0.05
- drift_window_threshold=300
- model_selector_decay=0.95
- Leaf model: $lr_w = lr_b = 0.01$, $L2 = 0$ (default fisso); seed=0
- StandardScaler online identico ai due lati

Due domande diverse

La configurazione deterministica misura la **fedeltà del port**; quella default misura la **qualità d'uso reale**.

Quattro risultati da verificare nei notebook

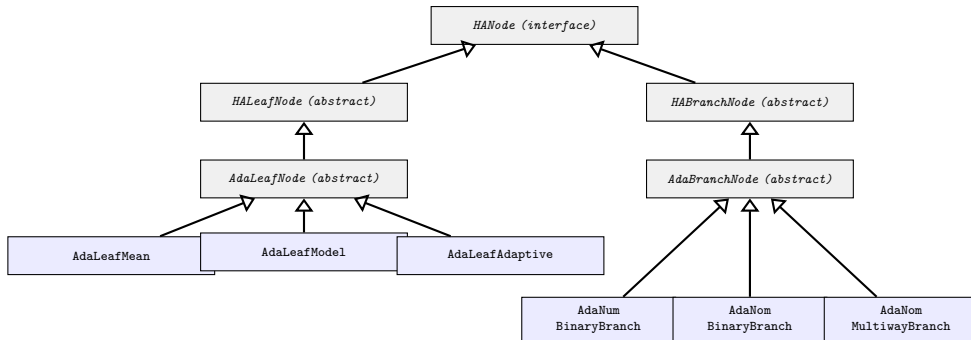
Domanda	Esperimento	Risultato
Il port replica River?	MEAN/MODEL/ADAPTIVE senza bootstrap	Predizioni e struttura coincidono numericamente
Il default mantiene la qualità?	ADAPTIVE con bootstrap	MAE e RMSE restano comparabili; le predizioni divergono per la RNG
Il training è più rapido?	8 dataset, cap a 17 000 istanze	Speedup complessivo osservato: circa 2.95–7.85×
Il modello è più compatto?	Stima finale del modello	Riduzione osservata: circa 2.2–11.9×

* Confronto indicativo: gli strumenti misurano heap Python e oggetti JVM con metodologie differenti.

Demo

I notebook mostrano dati, configurazioni, metriche e curve complete; questa tabella è la mappa con cui leggere la dimostrazione.

Struttura Java: gerarchia dei nodi



Separazione delle responsabilità: le foglie apprendono e propongono split; i branch instradano le istanze e gestiscono gli alberi alternativi.

Codice: apprendimento in una foglia adattiva

```
double y = inst.classValue();
double w = inst.weight();
double yPred = getPrediction(inst);           // pre-update

if (tree.bootstrapSampling) {
    int k = poissonSample(rng);               // Poisson(1)
    if (k > 0) w *= k;
}

double err = Math.abs(y - yPred);
driftDetector.update(err);                    // local ADWIN
errorTracker.update(err, 1.0);

updateModelSelector(inst, y, tree);           // faded MSE
baseLearnOne(inst, y, w);                     // stats + observer
trainLeafModel(inst, y, w);                   // SGD model

double seen = getTotalWeight();
if (seen - lastSplitAttemptAt >= tree.gracePeriod
    && isActive() && tree.growthAllowed) {
    tree.attemptToSplit(this, parent, parentBranch,
        tree.driftDetectorProto.createNew());
    lastSplitAttemptAt = seen;
}
```

Perché l'ordine conta

Predizione, errore e selezione del modello usano lo stato **prima** dell'aggiornamento: così non valutano il target appena osservato con un modello che lo ha già appreso.

Grace period

Gli observer vengono aggiornati a ogni istanza, ma la ricerca dello split parte soltanto ogni g unità di peso osservato.

Codice: dalla Hoeffding bound al nuovo branch

```
Collections.sort(candidates);
SplitCandidate best = candidates.get(size - 1);
SplitCandidate second = candidates.get(size - 2);

double n = leaf.getTotalWeight();
double eps = Math.sqrt(
    Math.log(1.0 / delta) / (2.0 * n));

boolean shouldSplit = best.merit > 0.0 && (
    second.merit / best.merit < 1.0 - eps
    || eps < tau);

if (best.isNullSplit()) {
    leaf.deactivate();
    return;
}

if (best.isNumeric)
    branch = new AdaNumBinaryBranch(...);
else if (best.isMultiway())
    branch = new AdaNomMultiwayBranch(...);
else
    branch = new AdaNomBinaryBranch(...);
```

Due condizioni di split

- il rapporto tra secondo e primo candidato è abbastanza piccolo;
- oppure $\varepsilon < \tau$, per rompere una situazione di parità.

Lo stesso risultato, tre strutture

Il tipo di SplitCandidate determina quale branch adattivo sostituisce la foglia, mantenendo statistiche e profondità.

Tipi di branch (split)

Tipi di branch (split)

- **AdaNumBinaryBranch**

split numerico: $x_i \leq t$; soglia da E-BST / TE-BST

- **AdaNomBinaryBranch**

split nominale binario: $x_i = v$ vs. tutto il resto

- **AdaNomMultiwayBranch**

un ramo per valore categorico; valore non visto $\rightarrow$ nuovo figlio dedicato (come River `add_child`); `maxBranches()` = -1

Perché non AdaNumMultiwayBranch?

In River esiste `AdaNumMultiwayBranchReg`, ma viene usata *solo* con `QOSplitter(allow_multiway_splits=True)`, configurazione non-default. Il default splitter (`TEBSTSplitter`) produce sempre split binari per attributi numerici $\Rightarrow$ `AdaNumBinaryBranch` è sufficiente per l'equivalenza con River HATR standard.

binary_split non implementato come opzione

River espone `binary_split=False` (default): per attributi nominali il multiway viene preferito; il binario lo sostituisce solo se ha merit strettamente superiore. Il Java replica questo comportamento in modo fisso in `HANominalObserver`: `binary_split=True` non è supportato.

Componenti di supporto

Observer per attributi numerici

- `HAEBSTObserver` — E-BST standard
- `HATEBSTObserver` — **Truncated** E-BST con arrotondamento *half-even* (`BigDecimal.HALF_EVEN`) per replicare `round(x, digits)` di Python

Utilities auto-contenute

- `VarStats` — varianza incrementale Welford (port di `river.stats.Var`)
- `ADWINDetector` — port fedele da River Cython
- `HAPerceptron` — modello lineare SGD
- `NormalDist` — CDF via Apache Commons Math 3 (`Erf.erfc`)

Dettagli implementativi non ovvi

Truncated EBST e arrotondamento

Il TEBST arrotonda i valori di split a un numero fisso di cifre decimali, riducendo i candidati unici nell'albero. Python `round()` usa banker's rounding (half-even); `Math.round()` in Java usa half-up. La discrepanza rompeva l'equivalenza: risolto usando `BigDecimal(x).setScale(d, HALF_EVEN)`.

Predizione con albero alternativo attivo

`traverseWithAlternate` non viene chiamato solo al momento dello swap: durante *tutta la vita* dell'albero alternativo, la predizione è la media tra il path principale e quello alternativo.

Attributi nominali: River vs. MOA

River — schema-less

Le feature arrivano come dizionari Python senza tipo dichiarato. `nominal_attributes` (lista di nomi) è l'unico modo per indicare al tree quali feature trattare come categoriche; senza, verrebbero passate al `TEBSTSplitter` numerico.

MOA — schema ARFF

Ogni dataset porta uno schema che dichiara il tipo di ogni attributo. `inst.attribute(i).isNominal()` funziona in automatico: nessuna configurazione richiesta all'utente.

merit_preprune: un caso limite utile

Meccanismo

Con `merit_preprune=True` il candidato *null split* ($\text{merit} = -\infty$) viene aggiunto alla lista. Se vince il test di Hoeffding la foglia viene **disattivata**.

Il null split viene selezionato solo se `len(candidates) < 2` (unico candidato). Con `len ≥ 2`, il controllo `best.merit > 0` fallisce prima ancora di raggiungere la selezione del candidato — il null non viene mai scelto.

Quando cambia davvero il comportamento

Con feature continue il null split non può diventare il migliore e l'opzione è normalmente ininfluente. Con una feature nominale ad alta cardinalità, invece, i branch possono avere merit zero:

- senza pre-pruning: split forzato anche senza segnale;
- con pre-pruning: il null split disattiva la foglia ed evita lo split spurio.

Test mirato: 60 foglie senza pre-pruning, 1 foglia con pre-pruning; River e CapyMOA coincidono a ogni campione.

La classe principale orchestra quattro responsabilità

Configurazione

Traduce le opzioni MOA in parametri operativi e crea i prototipi di observer e drift detector.

Predizione

Raccoglie le foglie raggiunte nel path principale e negli alberi alternativi, quindi ne media le predizioni.

Training

Inizializza la radice e delega ogni istanza a `AdaLeafNode` oppure `AdaBranchNode`.

Crescita e memoria

Decide quando creare uno split, aggiorna i contatori e applica periodicamente il limite di memoria.

Memory management

Ogni 10^6 istanze (configurabile), la classe principale esegue un ciclo di stima e controllo della memoria:

- 1 **Stima** (`estimateModelByteSizes`): misura la RAM reale via `SizeOf` e aggiorna le stime per foglia attiva e inattiva
- 2 **Controllo** (`enforceTrackerLimit`): se l'albero supera `maxByteSize`, ordina le foglie per *promise* (profondità) e **disattiva** le meno promettenti finché si rientra nel limite
- 3 **Potatura observer** (`pruneLeafObservers`): dopo un tentativo di split non riuscito, rimuove dagli EBST i tagli ormai troppo deboli rispetto al migliore

Foglia attiva vs. inattiva

Una foglia **inattiva** smette di aggiornare i propri attributi observer (risparmio di RAM), ma continua ad aggiornare statistiche e modello di predizione. Non può più splittare.

Se il budget lo permette, viene **riattivata**.

Opzione `stopMemManagement`

Se attiva, al superamento del limite si blocca direttamente `growthAllowed` invece di disattivare foglie: nessun nuovo split viene tentato.

`SizeOf` richiede l'agente JVM

`-javaagent:sizeofag.jar`; se assente, il meccanismo è silenziosamente disabilitato.

Dal drift allo swap: il ciclo in cinque passi

- 1 **Misura l'errore** Predizione del path principale e aggiornamento di $|y - \hat{y}|$.
- 2 **Interroga ADWIN** Una variazione significativa segnala un possibile drift.
- 3 **Crea l'alternativa** Una nuova foglia nasce alla stessa profondità e apprende in parallelo.
- 4 **Attende evidenza** Il confronto parte solo quando entrambi gli error tracker superano la soglia minima.
- 5 **Decide** Lo z-test produce uno *swap* oppure elimina l'alternativa.

Ordine degli aggiornamenti

L'istanza viene propagata sia all'albero alternativo sia al figlio corrente. I due modelli osservano quindi lo stesso stream dal momento in cui l'alternativa viene creata.

Codice: nascita e selezione dell'albero alternativo

1. Rilevazione e creazione

```
double err = Math.abs(y - yPred);
double oldMean = errorTracker.getMean();

driftDetector.update(err);
errorTracker.update(err, 1.0);
boolean drift = driftDetector.isDrift();

// Ignore changes while error improves
if (drift && errorTracker.getMean() < oldMean) {
    errorTracker = new VarStats();
    drift = false;
}

if (drift && alternateTree == null) {
    errorTracker = new VarStats();
    alternateTree = tree.newLeaf(null, depth);
    tree.nAlternateTrees++;
}
```

2. Z-test e decisione

```
double denom = Math.sqrt(
    altV / altN + curV / curN);
double z = (denom > 1e-12)
    ? (altMu - curMu) / denom : 0.0;
double p = 2.0 * NormalDist.cdf(-Math.abs(z));

if (p <= tree.switchSignificance) {
    if (altMu < curMu) {
        // Alternate wins: replace current subtree
        parent.children.set(parentBranch,
                             alternateTree);
        tree.nSwitchAlternateTrees++;
    } else {
        // Current subtree wins: discard alternate
        killNode(alternateTree, tree);
        alternateTree = null;
        tree.nPrunedAlternateTrees++;
    }
}
```

Il confronto viene eseguito solo quando entrambi gli error tracker superano driftWindowThreshold.

Compatibilità API: River → CapyMOA HATR

Parametro River	CapyMOA	
grace_period	grace_period	✓
delta	split_confidence	✓
tau	tie_threshold	✓
leaf_prediction	leaf_prediction	✓
model_selector_decay	model_selector_decay	✓
bootstrap_sampling	bootstrap_sampling	✓
min_samples_split	min_samples_split	✓
drift_window_threshold	drift_window_threshold	✓
switch_significance	switch_significance	✓
max_depth	max_depth	✓
merit_preprune	merit_preprune	✓
remove_poor_attrs	remove_poor_attrs	✓
stop_mem_management	stop_mem_management	✓
seed	random_seed	✓

Parametro River	Stato	
<i>Hardcoded al default River:</i>		
splitter	fisso TEBST	~
drift_detector	fisso ADWIN	~
leaf_model	fisso Perceptron	~
<i>Gestito automaticamente:</i>		
nominal_attrs	schema ARFF	✓
<i>Non implementato:</i>		
binary_split	solo False	×

✓ Opzioni principali espone nel wrapper.

~ Tre componenti fissati al default River.

× Una modalità non-default non implementata.

La configurazione standard è replicata. L'equivalenza puntuale vale senza bootstrap; con bootstrap si confrontano le metriche aggregate.

Cosa porta questo lavoro

1. Un learner nativo

HATR entra nell'ecosistema MOA/CapyMOA senza delegare il training a River.

2. Fedeltà verificabile

Nella configurazione deterministica, struttura e predizioni coincidono con River lungo tutto lo stream.

3. Beneficio prestazionale

Nel benchmark il tempo complessivo migliora di circa $2.95\text{--}7.85\times$, soprattutto grazie al training nella JVM.

4. Un confine chiaro

Il port replica il percorso standard, ma non rende ancora pluggabili splitter, drift detector e modello di foglia.

Risultato: HATR diventa confrontabile con gli altri learner streaming direttamente in CapyMOA.

Completamento parametri non-default

- 1 **binary_split=True**
Modificare HANominalObserver per imporre split binari anche su attributi nominali; aggiungere flag -B al learner MOA e parametro nel wrapper Python.
- 2 **Splitter pluggabile + AdaNumMultiwayBranch**
Implementare AdaNumMultiwayBranchReg e un HAQObserver per supportare QOSplitter(allow_multiway_splits=True); esporre splitter come opzione MOA.
- 3 **Leaf model pluggabile**
Aggiungere supporto per modelli leaf alternativi al Perceptron (es. regressione ridge).

Estensioni di architettura

- 4 **Drift detector pluggabile**
Definire un'interfaccia HADriftDetector e supportare detector alternativi ad ADWIN (es. EDDM, Page-Hinkley).

Riferimenti bibliografici I

- [1] P. Domingos and G. Hulten.
Mining high-speed data streams.
KDD, 2000.
- [2] A. Bifet and R. Gavaldà.
Learning from time-changing data with adaptive windowing.
SDM, 2007.
- [3] A. Bifet and R. Gavaldà.
Adaptive learning from evolving data streams.
IDA, 2009.
- [4] B. Babcock, M. Datar, R. Motwani, and L. O'Callaghan.
Maintaining variance and k-medians over data stream windows.
PODS, 2003.

Riferimenti bibliografici II

- [5] H. M. Gomes *et al.*
Adaptive random forests for evolving data stream classification.
Machine Learning, 2017.
- [6] E. Ikonomovska, J. Gama, and S. Dzeroski.
Learning model trees from evolving data streams.
Data Mining and Knowledge Discovery, 2011.
- [7] A. Bifet *et al.*
MOA: Massive Online Analysis.
JMLR, 2010.
- [8] C. Grzenda *et al.*
CapyMOA: A Python library for data stream mining.
arXiv, 2024.

Grazie per l'attenzione

Codice, notebook e JAR disponibili in:
`2025-2026_Regression-Models-in-CapyMOA/andrea_zhang/`

Implementazione:	<code>implementation/hatr-moa/</code>
Wrapper Python:	<code>implementation/hatr_capymoa/</code>
Esperimenti:	<code>experiments/</code>
Tutorial:	<code>tutorials/</code>